AceOS

Asset, Compliance & Equipment Operations System

Academic Project Report

Java Object-Oriented Programming and Relational Database Prototype

Project Detail	Value
Programming language	Java 24
Desktop interface	JavaFX 24.0.2
Database	MySQL 8.0
Database access	JDBC with MySQL Connector/J 26.7.0
Build and dependencies	Maven 3.9.16
Primary concepts	OOP, RDBMS, validation and transactions
Platform verified	Windows

Prepared from the completed AceOS prototype and verified source repository

4 October 2026

Table of Contents

Abstract 4

1. Introduction 4

1.1 Background . . . 4

1.2 Problem Statement . . . 4

1.3 Objectives . . . 4

1.4 Scope and Non-Goals . . . 4

1.5 Users and Use Cases . . . 5

2. Requirements and Development Methodology 5

2.1 Tested Configuration . . . 5

2.2 Functional Requirements . . . 5

2.3 Non-Functional Requirements . . . 5

2.4 Incremental Development Methodology . . . 5

3. System Design 6

3.1 Layered Architecture . . . 6

3.2 Responsibility of Each Layer . . . 6

3.3 Package Structure . . . 7

4. Object-Oriented Design and Core Data Model 7

4.1 UML-Style Domain Overview . . . 7

4.2 Model Responsibilities . . . 8

4.3 OOP Concepts Demonstrated . . . 8

5. Database Design 8

5.1 Normalized Schema . . . 8

5.2 Relationship Summary . . . 9

5.3 RDBMS Concepts . . . 9

6. Lifecycle and Transaction Design 9

6.1 State Transitions . . . 9
6.2 Atomic Corrective-Maintenance Operations . . . 10
6.3 Consistency Protection . . . 10
6.4 Preventive Maintenance and Inspection . . . 10
7. Major Modules and JavaFX Interface 10
7.1 Functional Modules . . . 10
7.2 Screen and Purpose Map . . . 11
7.3 Interface Architecture . . . 11
8. Validation, Error Handling and Security 11
8.1 Validation Strategy . . . 11
8.2 Error Handling . . . 11
8.3 Security and Data Protection . . . 11
9. Testing and Verification 12
9.1 Verification Approach . . . 12
9.2 Verification Matrix . . . 12
9.3 Latest Build Verification . . . 12
9.4 Known Test Harness Behaviour . . . 12
10. Results 12
11. Installation and Demonstration 13
11.1 Database Setup . . . 13
11.2 Environment Configuration . . . 13
11.3 Build and Launch . . . 13
11.4 Recommended Demonstration Flow . . . 13
12. Important Design Decisions 13
13. Challenges and Lessons Learned 14
14. Limitations and Future Scope 14
14.1 Current Limitations . . . 14

14.2 Possible Future Extensions 14

15. Conclusion **14**

16. Academic Relevance **15**

17. References **15**

Appendix A - Verification Programs **15**

Appendix B - Database Table Summary **15**

Appendix C - Repository and Release Details **16**

Appendix D - Dashboard Totals **16**

Appendix E - Formal Entity-Relationship Diagram **17**

Appendix F - Deterministic Demonstration Dataset **18**

Abstract

AceOS (Asset, Compliance & Equipment Operations System) is a local JavaFX desktop prototype for managing institutional assets, corrective maintenance, preventive-maintenance schedules and inspection compliance. It connects user and asset registration with complaint review, technician assignment, work-order execution, maintenance-record creation and administrative verification. Separate preventive-maintenance and inspection workflows provide recurring due-date management and compliance results.

The application is implemented in Java 24 with JavaFX 24.0.2. AceOsService coordinates application validation and use cases, domain models protect lifecycle transitions, JDBC repositories isolate SQL and transaction handling, and a normalized seven-table MySQL 8.0 schema preserves operational data. Searchable directories expose generated IDs and statuses, while one aggregate query supplies six live dashboard totals.

Repository, service and lifecycle verification programs protect valid and invalid workflow cases. A clean Maven package build on 4 October 2026 compiled all 58 Java source files successfully. The prototype demonstrates practical object-oriented design, relational modelling, referential integrity, prepared statements and multi-table transactions in one coherent academic system.

1. Introduction

1.1 Background

Institutional equipment such as projectors, air conditioners, generators, laboratory devices and safety equipment requires clear ownership of complaints, maintenance work, recurring service and inspection results. When records are fragmented, administrators cannot reliably determine which asset failed, who reported it, who performed the repair, whether the result was verified or when the next planned activity is due.

AceOS provides a simplified academic model of those operational concerns. It is not a production deployment; it is a desktop prototype that keeps its workflow rules explicit, its database normalized and its interface suitable for a complete classroom demonstration.

1.2 Problem Statement

A complaint alone is not an authorized maintenance task, and a completed repair should not automatically be considered verified. Planned maintenance and inspections also follow different lifecycles from corrective work. The project therefore requires a system that can preserve these distinctions, reject invalid state changes and update related database records consistently.

1.3 Objectives

- Register users with requester, technician and administrator roles.
- Register assets and track their operational status.
- Submit, accept or reject complaints and assign accepted complaints as work orders.
- Start, complete and administratively verify corrective maintenance.
- Create a maintenance record atomically when work is completed.
- Manage recurring preventive-maintenance schedules and next due dates.
- Schedule inspections and record passed or failed results.
- Provide searchable directories, status filters and live dashboard totals.
- Demonstrate OOP, JDBC, normalization, constraints, joins and transactions.

1.4 Scope and Non-Goals

The prototype covers a local, database-backed maintenance workflow from registration to verified completion. It does not provide login sessions, networked multi-user access, notifications, inventory procurement, document attachments, automatic work-order creation from failed inspections, general record deletion or production deployment.

1.5 Users and Use Cases

Role	Principal use cases
Requester	Reports a problem against a registered asset and identifies the originating user.
Technician	Receives work orders, performs maintenance and completes inspections.
Administrator	Reviews complaints, assigns work, manages schedules and verifies completed maintenance.

2. Requirements and Development Methodology

2.1 Tested Configuration

Component	Requirement / Tested Configuration
Operating system	Windows
Java	Java 24
Desktop toolkit	JavaFX 24.0.2
Build tool	Maven 3.9.16
Database	MySQL Server 8.0
JDBC driver	MySQL Connector/J 26.7.0
Configuration	ACEOS_DB_URL, ACEOS_DB_USER and ACEOS_DB_PASSWORD

2.2 Functional Requirements

- Persist and retrieve users, assets, complaints, work orders, maintenance records, schedules and inspections.
- Apply role, status, date, ID and required-text validation before persistence.
- Synchronize work-order, complaint and asset states during maintenance operations.
- Prevent duplicate work orders and duplicate maintenance records.
- Calculate schedule due dates, overdue schedules and inspection compliance results.
- Load database-backed directories and dashboard statistics through the service layer.

2.3 Non-Functional Requirements

- Integrity: domain rules, SQL constraints and expected-state updates must agree.
- Atomicity: related corrective-maintenance changes must commit or roll back together.
- Maintainability: UI, service, domain and repository responsibilities remain separated.
- Usability: forms return direct validation and database feedback, and directories expose required IDs.
- Credential safety: database secrets are supplied through environment variables rather than source code.
- Reproducibility: schema and representative sample-data scripts rebuild the database structure.

2.4 Incremental Development Methodology

The prototype was developed in verified increments rather than as one large implementation. Each stage added a small coherent layer and was checked before the next stage began.

Phase	Deliverable
1	Product concept, domain research and core workflow definition
2	Java domain entities, enums and lifecycle rules
3	Normalized MySQL schema, constraints and environment-based JDBC connection

Phase	Deliverable
4	Repositories, joins and relational object reconstruction
5	Transactional complaint, work-order and maintenance workflows
6	AceOsService validation and application-level error translation
7	JavaFX forms, navigation and database-backed directories
8	Preventive-maintenance and inspection-compliance workflows
9	Search, status filters, dashboard totals, documentation and release verification

3. System Design

3.1 Layered Architecture

AceOS separates presentation, application coordination, domain behaviour and persistence. JavaFX views do not execute SQL, and repositories do not contain user-interface logic. AceOsService is the application boundary used by every screen.

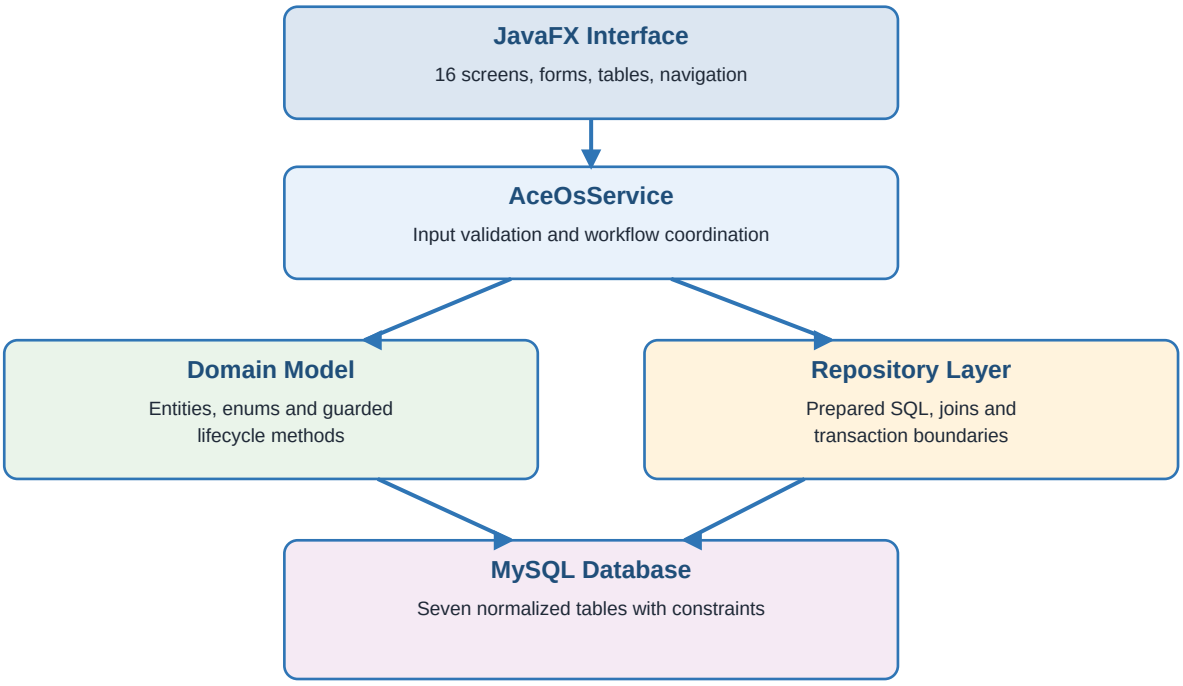


Figure 1. AceOS layered architecture and principal dependency direction.

3.2 Responsibility of Each Layer

Layer	Responsibility
JavaFX UI	Navigation, forms, tables, filters, feedback and dashboard presentation
AceOsService	Input validation, role checks, use-case coordination and database-error translation
Domain model	Entity state, associations, enums and guarded lifecycle transitions
Repositories	Prepared JDBC operations, joins, generated keys, expected-state updates and transactions
MySQL	Normalized storage, keys, uniqueness, checks and referential integrity

3.3 Package Structure

```
src/main/java/com/aceos
  database - environment-based JDBC connections
  model - entities, enums and lifecycle rules
  repository - persistence, joins and transactions
  service - application coordination and exceptions
  ui - JavaFX application shell and views
database - schema.sql and demo_reset.sql
```

4. Object-Oriented Design and Core Data Model

4.1 UML-Style Domain Overview

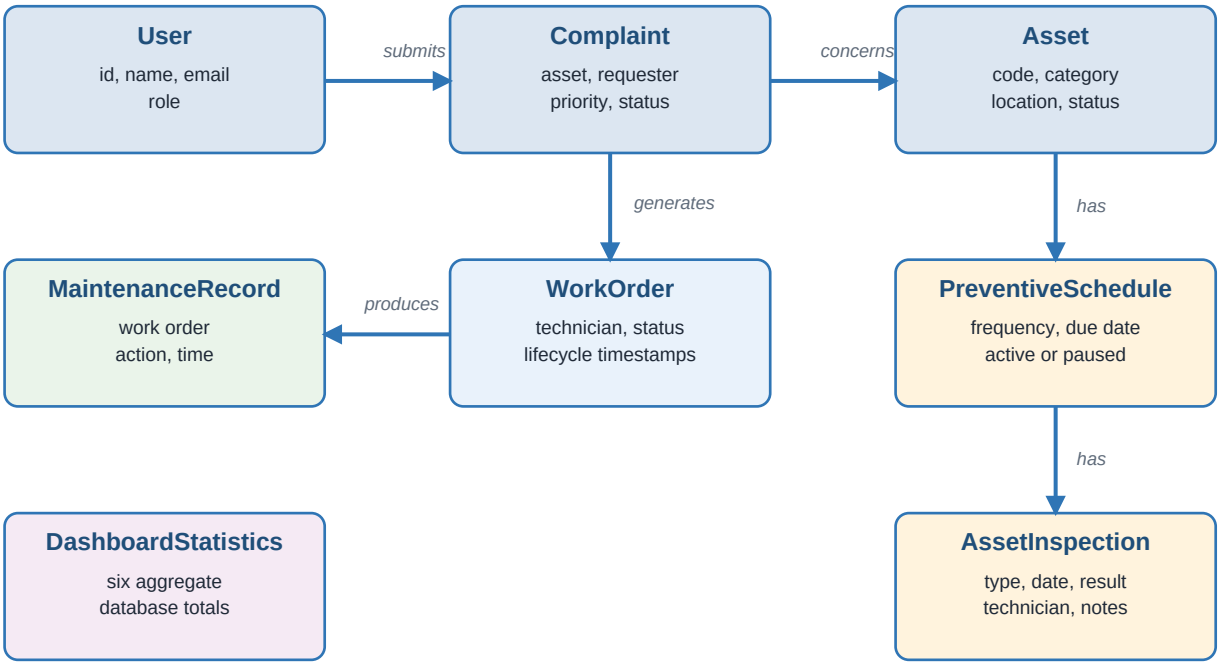


Figure 2. Principal model objects and their operational associations.

4.2 Model Responsibilities

Model	Primary responsibility
User	Stores identity, email and requester, technician or administrator role.
Asset	Represents the physical equipment, code, category, location and lifecycle status.
Complaint	Records an asset problem, requester, priority, review decision and resolution state.
WorkOrder	Represents assigned corrective work and its assignment, start, completion and verification times.
MaintenanceRecord	Stores the action taken and timestamp produced by work-order completion.
PreventiveMaintenanceSchedule	Stores a recurring task, frequency, next due date and active or paused state.
AssetInspection	Stores inspection type, schedule, technician, result, notes and completion date.
DashboardStatistics	Carries six aggregate totals from the database to the dashboard.

4.3 OOP Concepts Demonstrated

Concept	Implementation in AceOS
Encapsulation	Private fields and validating lifecycle methods protect entity state.
Composition / association	Complaints contain related asset and requester objects; work orders contain complaint and technician objects.
Inheritance	AceOsApplication extends JavaFX Application, and view classes extend JavaFX layout controls.
Abstraction	AceOsService hides workflow coordination and repositories hide JDBC details.
Polymorphism	JavaFX lifecycle and control behaviour are used through inherited and overridden APIs.
State modelling	Enums represent roles, priorities and valid workflow states.
Exception handling	AceOsServiceException exposes clear application-level validation and persistence failures.

5. Database Design

5.1 Normalized Schema

The MySQL schema separates identity, assets and each operational workflow into seven tables. Repositories reconstruct connected Java objects through joins, preserving normalization without forcing the interface to work with disconnected foreign-key values.

Table	Important columns	Relationships / constraints
users	user_id, name, email, role	Unique email; constrained role
assets	asset_id, asset_code, name, category, location, status	Unique asset code; constrained status
complaints	asset_id, requester_id, description, priority, status, reported_at	Many complaints per asset/requester
work_orders	complaint_id, technician_id, status, lifecycle timestamps	Unique complaint; one technician per order
maintenance_records	work_order_id, action_taken, recorded_at	Unique work order; zero or one final record

Table	Important columns	Relationships / constraints
preventive_maintenance_schedules	asset_id, task, frequency_days, next_due_date, status	Positive frequency; many schedules per asset
asset_inspections	asset_id, type, dates, status, technician_id, notes	Completion fields constrained by status

5.2 Relationship Summary

Relationship	Cardinality
Asset to Complaint	One to many
Requester to Complaint	One to many
Complaint to WorkOrder	One to zero-or-one
Technician to WorkOrder	One to many
WorkOrder to MaintenanceRecord	One to zero-or-one
Asset to PreventiveMaintenanceSchedule	One to many
Asset to AssetInspection	One to many

5.3 RDBMS Concepts

- Primary keys uniquely identify every persisted record.
- Foreign keys retain valid relationships and restrict unsafe parent updates or deletions.
- Unique constraints prevent duplicate user emails, asset codes, work orders and maintenance records.
- CHECK constraints protect roles, priorities, statuses, schedule frequency and inspection completion consistency.
- Prepared statements bind input values and generated keys return database identities.
- Joins rebuild related object graphs for work orders, records, schedules and inspections.

6. Lifecycle and Transaction Design

6.1 State Transitions

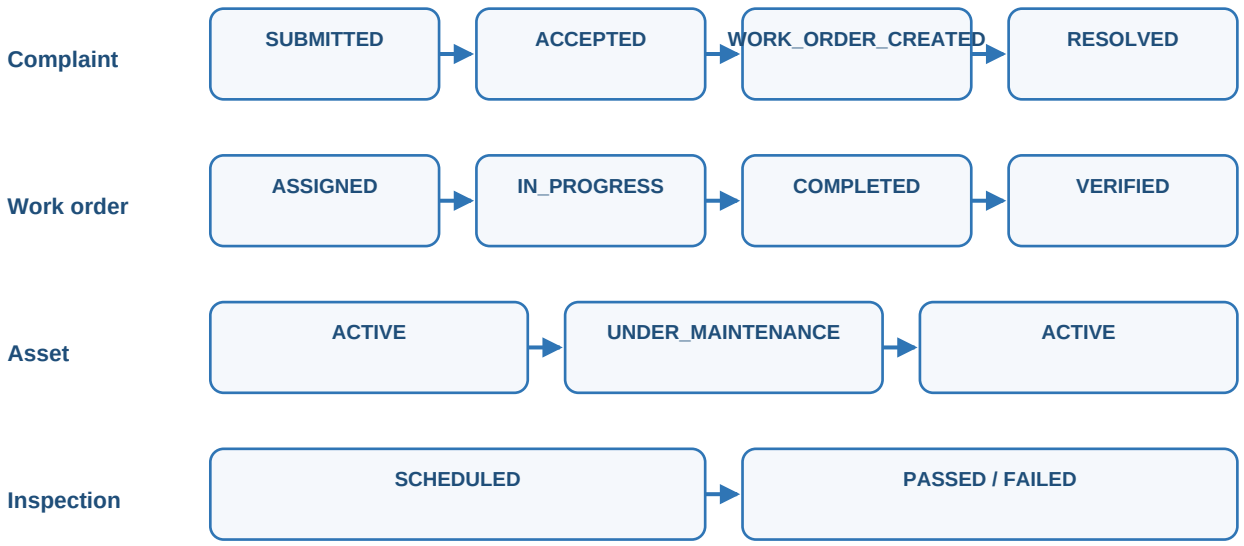


Figure 3. Principal lifecycle transitions. Invalid or repeated transitions are rejected.

A submitted complaint may also move directly to REJECTED. A preventive schedule moves between ACTIVE and PAUSED; recording completion while active advances the next due date by the configured frequency while keeping the schedule active.

6.2 Atomic Corrective-Maintenance Operations

Operation	Atomic database changes
Create work order	Insert the assigned work order and change the complaint from ACCEPTED to WORK_ORDER_CREATED.
Start work	Change the order from ASSIGNED to IN_PROGRESS and the asset from ACTIVE to UNDER_MAINTENANCE.
Complete work	Change the order from IN_PROGRESS to COMPLETED and insert the maintenance record.
Verify work	Change the order to VERIFIED, the complaint to RESOLVED and the asset back to ACTIVE.

6.3 Consistency Protection

Each multi-record operation disables auto-commit, performs every expected update and commits only after complete success. SQL WHERE clauses include the expected prior state, so repeated or stale operations affect zero rows and cause rollback. This complements the domain methods, which reject illegal state changes before persistence.

6.4 Preventive Maintenance and Inspection

Preventive maintenance is deliberately independent of complaints. A schedule stores a task, frequency and next due date; it can be paused, reactivated or completed. Inspection completion is limited to technicians, requires notes and records PASSED or FAILED with a completion date. Failed inspections are shown in dashboard and directory results but do not automatically create corrective work orders.

7. Major Modules and JavaFX Interface

7.1 Functional Modules

Module	Purpose
User and Role Management	Register requester, technician and administrator users and expose generated IDs.
Asset Management	Register physical assets and show location and lifecycle status.
Complaint Management	Submit complaints, review submitted cases and accept or reject them.
Work Order Management	Assign accepted complaints, start work, complete work and verify the result.
Maintenance Records	Persist the action taken when corrective work is completed.
Preventive Maintenance	Create schedules, pause/reactivate them and record recurring completion.
Inspection Tracking	Schedule inspections and record passed or failed technician results.
Directories	Expose operational IDs, related data, statuses, search and filters.
Dashboard	Display six live aggregate totals and refresh them from one repository query.

7.2 Screen and Purpose Map

Screen	Purpose
Dashboard	Total and active assets, open complaints, active work orders, overdue schedules and failed inspections
Users / User Directory	Register users and locate IDs, names, emails and roles
Assets / Asset Directory	Register assets and locate IDs, codes, locations and statuses
Complaints / Complaint Directory	Submit complaints and browse reported details and IDs
Review Complaints	Accept or reject submitted complaints
Work Orders / Work Order Directory	Assign technicians and browse assignment/lifecycle details
Maintenance	Start and complete work orders and create maintenance records
Verify Maintenance	Perform administrator verification of completed work
Manage Schedules / Schedule Directory	Create and operate recurring preventive schedules
Manage Inspections / Inspection Directory	Schedule and complete inspection results

7.3 Interface Architecture

AceOsApplication builds a BorderPane shell with a shared header, a scrollable sidebar and a replaceable center view. One shared AceOsService instance is passed to every screen. The interface is programmatic JavaFX and does not use FXML.

Directory screens load records from the database and use JavaFX FilteredList for immediate client-side search. Asset, complaint, work-order, schedule and inspection directories also provide lifecycle-status filters. Form screens parse IDs, validate inputs and display success, validation or database feedback without exposing SQL details.

8. Validation, Error Handling and Security

8.1 Validation Strategy

Validation area	Examples
Text	Required names, descriptions, actions and notes; maximum-length checks
Identifiers	Whole-number parsing, positive IDs and existence checks
Roles	Requester required for complaints, technician required for work/inspections, administrator required for verification
Dates	Schedule due date cannot begin in the past; completion date cannot be in the future
Lifecycle	Only legal prior states may transition; repeated operations are rejected
Database	Unique and foreign-key errors are translated into application-level messages

8.2 Error Handling

AceOsService catches database failures and converts them into AceOsServiceException messages suitable for the interface. Domain IllegalStateException and IllegalArgumentException conditions are also translated at the application boundary. Repository transactions attach rollback failures as suppressed exceptions rather than hiding the original cause.

8.3 Security and Data Protection

- Database URL, username and password are supplied through environment variables.
- Prepared statements separate SQL from bound input values.

- Foreign keys and constraints protect relational consistency.
- The source package excludes real credentials and current live database records.
- Roles are validated during relevant operations, but the prototype does not provide login sessions or screen-level authorization.

9. Testing and Verification

9.1 Verification Approach

AceOS uses executable verification classes for repository, service and lifecycle checks rather than a JUnit suite. Database-backed checks require the three database environment variables and a running MySQL server. JavaFX behaviour was verified manually.

9.2 Verification Matrix

Verification area	Representative evidence	Result
Clean package build	58 Java source files compiled; JAR created	PASS
Database connection	Environment-based connection and clear missing-variable failure	PASS
User and asset repositories	Generated IDs, retrieval and uniqueness constraints	PASS
Complaint repository	Submission, joins, acceptance and rejection protection	PASS
Work-order repository	Assignment, relational reconstruction and complaint synchronization	PASS
Maintenance lifecycle	Start, complete, record creation, verify and repeated-action rejection	PASS
AceOsService validation	Role, ID, text, date, state and database-error translation	PASS
Preventive maintenance	Create, retrieve, pause, activate, complete and due-date advancement	PASS
Asset inspections	Schedule, technician completion, pass/fail results and invalid-operation protection	PASS
Directories and dashboard	Database retrieval, search/filter behaviour and six aggregate totals	PASS

9.3 Latest Build Verification

A clean build was rerun on 4 October 2026 with `mvn clean package`. Maven reported BUILD SUCCESS, compiled 58 source files for Java 24 and created `target/aceos-1.0.0.jar`. The package phase does not automatically execute the standalone database verification programs.

9.4 Known Test Harness Behaviour

Some database verification runs launched through Maven exec may print a non-fatal MySQL Connector/J abandoned-connection-cleanup thread warning after BUILD SUCCESS. The reported repository and lifecycle checks still pass; converting the verification harness to JUnit with explicit test-lifecycle cleanup remains future work.

10. Results

The completed prototype supports the entire corrective-maintenance route from registration and complaint submission through assignment, maintenance completion, record creation and administrative verification. The final states remain synchronized as VERIFIED work order, RESOLVED complaint and ACTIVE asset.

Preventive maintenance and inspection compliance operate as separate persisted workflows. Six directories expose the IDs and statuses needed by forms, all directories support search, lifecycle directories support status filtering, and the dashboard retrieves six current totals in one query. The application builds successfully, the repository is clean and the source is released under the v1.0.0 Git tag.

11. Installation and Demonstration

11.1 Database Setup

- Install Java 24, Maven 3.9 or newer and MySQL Server 8.0.
- Execute database/schema.sql in MySQL Workbench or another MySQL client.
- Execute database/demo_reset.sql whenever a clean demo state is required.
- Create or select a MySQL application user with SELECT, INSERT and UPDATE permission on the aceos database.

11.2 Environment Configuration

```
$env:ACEOS_DB_URL = "jdbc:mysql://localhost:3306/aceos"
$env:ACEOS_DB_USER = "aceos_app"
$env:ACEOS_DB_PASSWORD = "<local database password>"
```

The variables must be set in the same PowerShell session used to launch AceOS. Real credentials must never be added to source control.

11.3 Build and Launch

```
mvn clean package
mvn javafx:run
```

11.4 Recommended Demonstration Flow

1. Register a requester, technician and administrator; note their generated IDs in User Directory.

2. Register an asset; note its generated ID and ACTIVE status in Asset Directory.

3. Submit a complaint using the requester and asset IDs.

4. Accept the complaint administratively and assign its ID to the technician as a work order.

5. Start the work order and confirm that the asset becomes UNDER_MAINTENANCE.

6. Complete the work with an action description and note the generated maintenance-record ID.

7. Verify the completed work using the administrator ID; confirm VERIFIED, RESOLVED and ACTIVE states.

8. Create, pause, reactivate and complete a preventive schedule; confirm the next due date advances.

9. Schedule an inspection and complete it as PASSED or FAILED using the technician ID.

10. Use search, status filters and dashboard refresh to demonstrate operational visibility.

12. Important Design Decisions

- User roles are enum values on one User class rather than separate requester, technician and administrator subclasses.
- A work order reaches its asset through the complaint so corrective work remains tied to the original report.
- Work-order completion and maintenance-record creation share one transaction.
- Administrative verification is separate from technician completion and synchronizes three records.
- Preventive schedules and inspections are independent of complaints because they can exist before a failure occurs.
- Expected-state SQL updates provide database-level protection against repeated or stale actions.
- Directories expose operational IDs so the demonstration does not depend on manual database lookup.
- The dashboard uses one aggregate query so the view contains no JDBC logic.

13. Challenges and Lessons Learned

Challenge	Lesson
Coordinating related lifecycle states	Model the legal transition first, then persist every related change atomically.
Reconstructing normalized records	SQL joins can preserve database normalization while returning complete domain objects.
Preventing repeated actions	Domain validation alone is insufficient; expected-state WHERE clauses protect the stored state.
Using generated IDs in forms	Database-backed directories make IDs visible and reduce demonstration errors.
Separating UI and persistence	A service boundary keeps JavaFX event handlers small and testable.
Managing database credentials	Environment variables avoid hard-coded secrets but must be set in the launch terminal.

14. Limitations and Future Scope

14.1 Current Limitations

- Local JavaFX desktop prototype rather than a deployed web or networked system.
- No login, session or screen-level authorization.
- MySQL Server must run separately and the environment variables must be configured.
- Standalone executable verification programs are not integrated into mvn test as JUnit tests.
- No general audit-log screen, notifications, attachments, certificate storage or report export.
- Schedule completion and failed inspections do not automatically create corrective work orders.
- The release package contains source and schema scripts, not the current live database records.

14.2 Possible Future Extensions

- Add login and role-based sessions using the existing role model.
- Convert verification programs into repeatable JUnit integration tests.
- Add persistent activity logging and reminders for overdue work.
- Add maintenance-history browsing, report export and supporting attachments.
- Allow failed inspections or due schedules to create controlled corrective work requests.
- Consider network deployment and multi-organization support only if requirements expand.

15. Conclusion

AceOS demonstrates how object-oriented domain models and a relational database can cooperate in a practical maintenance-management problem. Encapsulated entities protect valid lifecycle changes, the service layer coordinates use cases, repositories isolate JDBC and transactions, and MySQL constraints preserve stored integrity. JavaFX provides a complete desktop interaction layer without mixing presentation code with SQL.

The prototype is functionally complete for its defined academic scope. It supports corrective, preventive and inspection workflows, exposes operational records through searchable directories, reports live dashboard totals and can be recreated from its tracked schema and source package.

16. Academic Relevance

The project directly applies Java classes, objects, enums, encapsulation, inheritance, abstraction, polymorphism, composition and exception handling. It also demonstrates normalization, primary and foreign keys, cardinality, constraints, joins, generated keys, prepared statements, transactions, rollback, aggregation and environment-based configuration. These concepts appear as necessary design choices in one working system rather than isolated exercises.

17. References

1. Oracle. Java Platform, Standard Edition 24 documentation. https://docs.oracle.com/en/java/javase/24/

2. OpenJFX. JavaFX documentation. https://openjfx.io/

3. Oracle. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

4. Apache Software Foundation. Apache Maven documentation. https://maven.apache.org/guides/

5. AceOS source repository, README.md, database/schema.sql and Java source files. https://github.com/SmitHitnalli/aceos

Appendix A - Verification Programs

The following executable classes provide the recorded repository, service and lifecycle verification coverage:

- UserRepositoryTest and AssetRepositoryTest
- ComplaintRepositoryTest and WorkOrderRepositoryTest
- WorkOrderLifecycleTest and MaintenanceRecordRepositoryTest
- AceOsServiceTest and AceOsServiceValidationTest
- PreventiveMaintenanceScheduleRepositoryTest
- PreventiveMaintenanceServiceTest and PreventiveMaintenanceLifecycleTest
- AssetInspectionServiceTest

Appendix B - Database Table Summary

Table	Purpose
users	Application identities and role values
assets	Physical equipment and lifecycle status
complaints	Reported problems and administrative review state
work_orders	Corrective assignments and lifecycle timestamps
maintenance_records	Action taken for completed corrective work
preventive_maintenance_schedules	Recurring tasks, frequencies and due dates
asset_inspections	Scheduled checks and pass/fail compliance results

Appendix C - Repository and Release Details

Item	Value
Repository	https://github.com/SmitHitnalli/aceos
Release reference	GitHub v1.0.0 tag
Git tag / release	v1.0.0
Source package	AceOS-v1.0.0-source.zip
Release details	Available on the GitHub release page
Current Maven coordinate	com.aceos:aceos:1.0.0
Latest verified build	BUILD SUCCESS - 4 October 2026

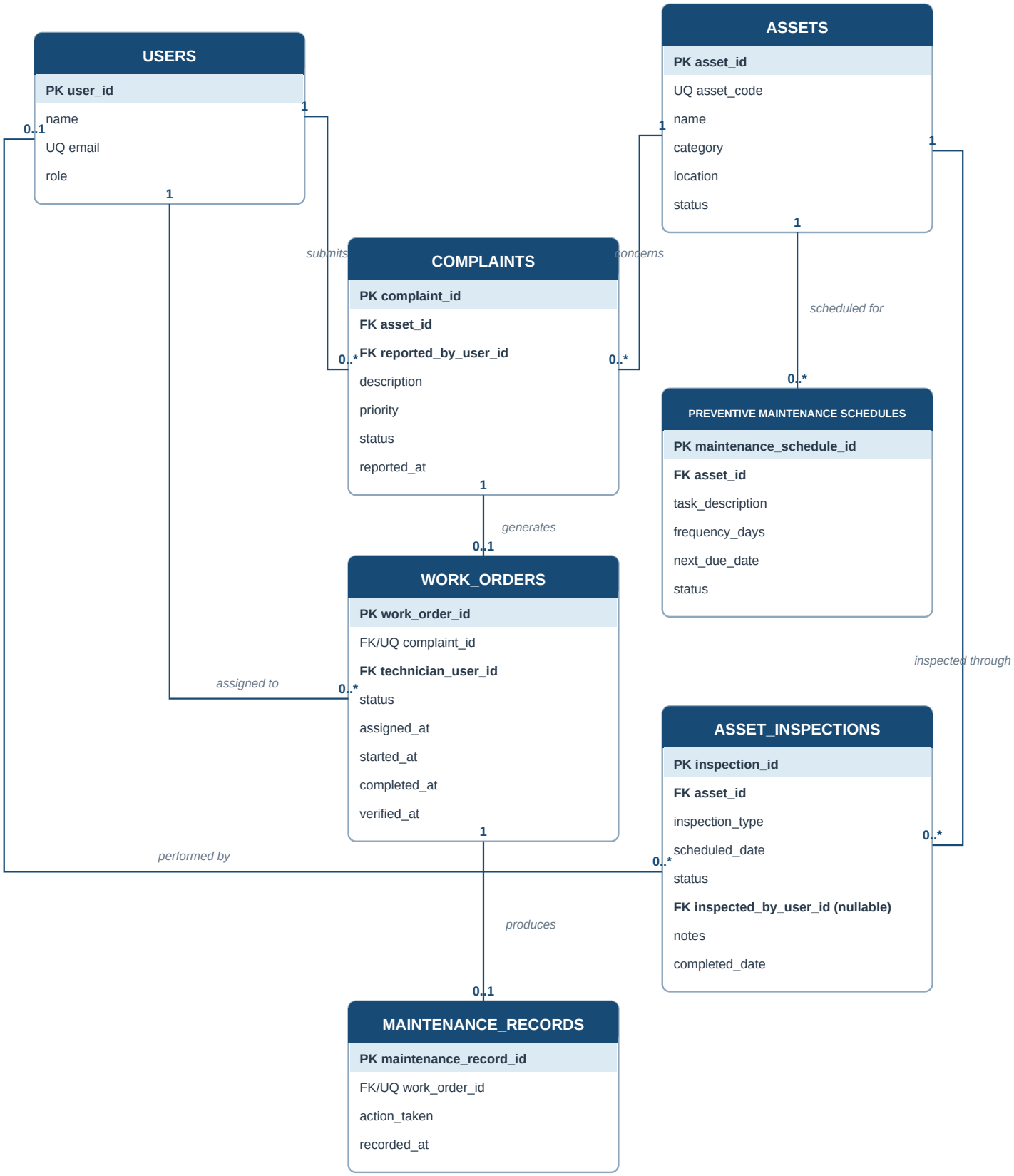
The Maven project coordinate, generated JAR name, Git tag and GitHub release now use the same 1.0.0 release version.

Appendix D - Dashboard Totals

Metric	Definition
Total assets	All rows in assets
Active assets	Assets whose status is ACTIVE
Open complaints	Complaints not REJECTED or RESOLVED
Active work orders	Work orders whose status is not VERIFIED
Overdue schedules	ACTIVE schedules with a next due date before the current date
Failed inspections	Inspections whose status is FAILED

Appendix E - Formal Entity-Relationship Diagram

The diagram maps the seven persisted tables, their primary and foreign keys, and implemented cardinalities.



Legend: PK = primary key; FK = foreign key; UQ = unique. 1, 0..1 and 0..* show relationship cardinality.

Appendix F - Deterministic Demonstration Dataset

database/demo_reset.sql rebuilds a complete, predictable AceOS demonstration state.

Demonstration database only

The script deletes all existing AceOS rows. Run it through an administrative MySQL connection, never against a database whose records must be retained.

Canonical reset procedure

1. Execute database/schema.sql once to create the database and seven tables.
2. Execute database/demo_reset.sql before a demonstration or whenever a clean state is required.
3. Confirm that the final query reports PASS and the six expected dashboard totals.

Stable demonstration IDs

Role	ID
Requester	1
Technician	2
Administrator	3

Workflow record	ID
Submitted complaint	1
Accepted complaint	2
Assigned work order	1
In-progress work order	2
Completed work order	3
Verified work order	4

Lifecycle coverage

Complaints: SUBMITTED, ACCEPTED, WORK_ORDER_CREATED, RESOLVED and REJECTED.

Work orders: ASSIGNED, IN_PROGRESS, COMPLETED and VERIFIED, with required maintenance records.

Schedules: overdue active, upcoming active and paused. Inspections: SCHEDULED, PASSED and FAILED.

Expected dashboard totals after every reset

Dashboard metric	Expected total
Total assets	6
Active assets	4
Open complaints	5
Active work orders	3
Overdue schedules	1
Failed inspections	1

Determinism rule

Explicit IDs keep form inputs stable; relative dates keep overdue and upcoming categories meaningful.
Running the script repeatedly must return the same IDs, lifecycle coverage and dashboard totals.